ICMC USP SÃO CARLOS



Análise Assintótica

Analisando Assintoticamente Algoritmos



Sumário

- Propriedades Assintóticas
- Conceitos Importantes
- Analisando Assintoticamente

Referências

- Aula baseado no capítulo “Introdução à Análise Assintótica” de SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al. Análise de Algoritmos. Porto Alegre: SAGAH, 2021. E-book. ISBN 9786556901862. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9786556901862/
- CORMEN, Thomas. Algoritmos - Teoria e Prática. 3rd ed. Rio de Janeiro: GEN LTC, 2012. E-book. p.32. ISBN 9788595158092. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9788595158092/

Propriedades

- Transitividade:

  $f(n)=O(g(n))$ e $g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$

  $f(n)=\Omega(g(n))$ e $g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n))$

  $f(n)=\Theta(g(n))$ e $g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$

- Reflexividade:

  $f(n) = O(f(n))$

  $f(n) = \Omega(f(n))$

  $f(n) = \Theta(f(n))$

Propriedades

- Simetria:

  $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$

- Simetria de transposição:

  $f(n) = O(g(n))$ se e somente se $g(n) = \Omega(f(n))$,

  $f(n) = o(g(n))$ se e somente se $g(n) = \omega(f(n))$.

Propriedades

- Reflexividade:

  $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$

  $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$

- Teorema:

  $f(n) = O(g(n))$ e $f(n) = \Omega(g(n)) \Rightarrow f(n) = \Theta(g(n))$

Propriedades

- Analogia entre comparação assintótica de funções f(n) e g(n) com a comparação de números a e b reais:

  $f(n)=O(g(n))$ é como $a \leq b$.

  $f(n)=\Omega(g(n))$ é como $a \geq b$.

  $f(n)=\Theta(g(n))$ é como $a = b$.

  $f(n)=o(g(n))$ é como $a < b$.

  $f(n)=\omega(g(n))$ é como $a > b$.

Propriedades

- f(n) é assintoticamente menor que g(n) se f(n) = o(g(n)).
- f(n) é assintoticamente maior que g(n) se f(n) = ω(g(n)).
  - Podemos comparar quaisquer dois números reais, mas não podemos comparar assintoticamente todas as funções.

Conceitos Importantes

- Uma função f(n) é monotonicamente crescente se $m \leq n \Rightarrow f(m) \leq f(n)$.
- Uma função f(n) é monotonicamente decrescente se $m \leq n \Rightarrow f(m) \geq f(n)$

.

- Uma função f(n) é estritamente crescente se $m < n \Rightarrow f(m) < f(n)$.
- Uma função f(n) é estritamente decrescente se $m < n \Rightarrow f(m) > f(n)$.

Conceitos Importantes

- Função Piso:

  $\forall x \in \mathfrak{R}$, o maior inteiro menor que ou igual a x é denotado por $\lfloor x \rfloor$.

- Função teto

  $\forall x \in \mathfrak{R}$, o menor inteiro maior que ou igual a x é denotado por $\lceil x \rceil$.

- Temos que:
  - $\forall x \in \mathfrak{R}$, $x-1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x+1$.
  - As duas funções são monotonicamente crescentes.

Conceitos Importantes

- Aritmética modular: Se a e b são inteiros e n é um inteiro positivo, então a é congruente com b módulo n se n divide a–b.
  - $a \equiv b \pmod{n}$
- $a \equiv b \pmod{n}$ se a e b têm o mesmo resto quando divididos por n
- $0 \leq b \bmod n < n$

Conceitos Importantes

- Aritmética modular: Se a e b são inteiros e n é um inteiro positivo, então a é congruente com b módulo n se n divide a–b.
  - $a \equiv b \pmod{n}$
- $a \equiv b \pmod{n}$ se a e b têm o mesmo resto quando divididos por n
- $b \bmod n = b - b \lfloor b/n \rfloor$.
- $0 \leq b \bmod n < n$

Conceitos Importantes

- Exemplo: $17 \equiv 5 \bmod 6$

  a=17, b=5 e n=6

  $\Rightarrow$ 6 divide 12 = 17-5

  $\Rightarrow$ 17 tem resto 5 quando dividido por 6

  $\Rightarrow$ 5 tem resto 5 quando dividido por 6

Analisando Assintoticamente

```
1   def subConjunto(L1, L2):
2     '''
3     Entrada: L1, L2
4     Saída: True or False, Se L1 eh subconjunto de L2
5     '''
6     for i in L1:
7       estaContido = False
8       for j in L2:
9         if i == j:
10          estaContido = True
11          break
12      if not estaContido:
13          return False
14    return True
```

O(len(L1)*len(L2))

O(n*n)=O(n^2)

O(len(L1))

O(len(L2))

Analisando Assintoticamente

```
1   def intersec(L1, L2):
2     """Entrada: L1 e L2
3        Saída: Lista com elementos em comum"""
4     #Armazena elementos em comum
5     tmp = []
6     for i in L1:
7       for j in L2:
8         if i == j:
9           tmp.append(i)
10    #Remove duplicatas
11    intersec = []
12    for i in tmp:
13      if i not in intersec:
14        intersec.append(i)
15    return intersec
```

O(len(L1)*len(L2))
$O(n^2)$

O(len(tmp)*len(intersec))
$O(n^2)$

$O(n^2)=O(n^2)+O(n^2)$

Analisando Assintoticamente

Número de Divisões

O código abaixo dividi o valor de x, inteiro positivo, por 2 até que x fique menor ou igual a 1. Seja x = n, teremos:

```
x = 100
while(x>1):
    x = x//2
```

$$\frac{n}{2} - - - 1^a \text{ divisão}$$

$$\downarrow$$

$$\frac{n}{2^2} - - - 2^a \text{ divisão}$$

$$\downarrow$$

$$\frac{n}{2^3} - - - 3^a \text{ divisão}$$

$$\downarrow$$

$$\vdots$$

$$\frac{n}{2^m} - - - \text{m-ésima divisão}$$

Analisando Assintoticamente

O código abaixo dividi o valor de x, inteiro positivo, por 2 até que x fique menor ou igual a 1. Seja x = n, teremos:

```
x = 100
while(x>1):
    x = x//2
```

$\frac{n}{2}$ Número de Divisões

$\downarrow$

$\frac{n}{2^2} \Rightarrow \frac{n}{2^m} \leq 1$

$\downarrow$

$\frac{n}{2^3} \qquad 2^m \geq n$

$\downarrow \qquad log_2 2^m \geq log_2 n$

$\vdots \qquad m \geq log_2 n \Rightarrow m = \lceil log_2 n \rceil$

$\frac{n}{2^m}$

Analisando Assintoticamente

O código abaixo dividi o valor de x, inteiro positivo, por 2 até que x fique menor ou igual a 1. Seja x = n, teremos:

Número de Divisões

Número de Iterações

```
x = 100
while(x>1):
    x = x//2
```

$$O(\lceil log_2 n \rceil)$$

$$\lceil log_2 n \rceil \leq log_2 n + 1$$

$$O(\lceil log_2 n \rceil) \leq O(log_2 n + 1) = O(log_2 n)$$

Complexidade $O(log_2 n)$

Analisando Assintoticamente

Converte inteiro para string

```python
def converte_int_str(i):
    """Entrada: n inteiro positivo
       Saída: Uma representação decimal em string de n"""
    digitos='0123456789'
    if i == 0:
        return '0'
    string_inteira = ''
    while i > 0:
        string_inteira = digitos[i%10] + string_inteira
        i = i//10
        print(f'{string_inteira=} {i=}')
    return string_inteira
```

```python
converte_int_str(12345678)
```

```
string_inteira='8' i=1234567
string_inteira='78' i=123456
string_inteira='678' i=12345
string_inteira='5678' i=1234
string_inteira='45678' i=123
string_inteira='345678' i=12
string_inteira='2345678' i=1
string_inteira='12345678' i=0
'12345678'
```

Analisando Assintoticamente

Converte inteiro para string

```python
def converte_int_str(i):
    """Entrada: n inteiro positivo
       Saída: Uma representação decimal em string de n"""
    digitos='0123456789'
    if i == 0:
        return '0'
    string_inteira = ''
    while i > 0:
        string_inteira = digitos[i%10] + string_inteira
        i = i//10
        print(f'{string_inteira=} {i=}')
    return string_inteira
```

$\frac{i}{10}$ — — — 1ª divisão

↓

$\frac{i}{10^2}$ — — — 2ª divisão

↓

$\frac{i}{10^3}$ — — — 3ª divisão

↓

⋮

$\frac{i}{10^m}$ — — — m-ésima divisão

Analisando Assintoticamente

Converte inteiro para string

```python
def converte_int_str(i):
    """Entrada: n inteiro positivo
       Saída: Uma representação decimal em string de n"""
    digitos='0123456789'
    if i == 0:
        return '0'
    string_inteira = ''
    while i > 0:
        string_inteira = digitos[i%10] + string_inteira
        i = i//10
        print(f'{string_inteira=} {i=}')
    return string_inteira
```

$$\Rightarrow \frac{i}{10^m} = 1$$

$$10^m = i$$

$$log(10^m) = log(i)$$

$$m = log(i)$$

$$\Rightarrow \text{Complexidade } O(log(i))$$

De fato, a complexidade é

$$\Theta(log(i))$$